SCF_assignment3_solutions

April 22, 2026

[1]:
```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import scipy
```

0.0.1 Exercise 1a

Let us compute v_3. Since $\mathbb{S}_3$, the stock price at time 3, is completely determined by the information in $\mathcal{A}_3$, we express $\mathbb{S}_4$ in terms of $\mathbb{S}_3$ as $\mathbb{S}_4 = u\mathbb{S}_3$ or $d\mathbb{S}_3$ if the fourth coin toss is H or T respectively. This gives the conditional expectation

$$\tilde{\mathbb{E}}\left[\frac{v_4(\mathbb{S}_4)}{(1+r)^4}\middle|\mathcal{A}_3\right] = \frac{1}{(1+r)^4}\left(\tilde{p}v_4(u\mathbb{S}_3) + \tilde{q}v_4(d\mathbb{S}_3)\right) \ ,$$

which in turn implies that

$$v_3(x) = \frac{1}{1+r}\left(\tilde{p}v_4(ux) + \tilde{q}v_4(dx)\right) \ , \tag{1}$$

with x substituted by $\mathbb{S}_3$. Next, we can determine $v_2(\mathbb{S}_2)$ in terms of $v_3(\mathbb{S}_3)$ similarly:

$$\begin{aligned} v_2(x) &= \frac{1}{1+r}\left(\tilde{p}v_3(ux) + \tilde{q}v_3(dx)\right) \ , \\ &= \frac{1}{(1+r)^2}\left(\tilde{p}^2v_4(u^2x) + 2\tilde{p}\tilde{q}v_4(udx) + \tilde{q}^2v_4(d^2x)\right) \ , \end{aligned} \tag{2}$$

with x substituted by $\mathbb{S}_2$. Next, v_1 is determined as

$$\begin{aligned} v_1(x) &= \frac{1}{1+r}\left(\tilde{p}v_2(ux) + \tilde{q}v_2(dx)\right) \ , \\ &= \frac{1}{(1+r)^3}\left(\tilde{p}^3v_4(u^3x) + 3\tilde{p}^2\tilde{q}v_4(u^2dx) + 3\tilde{p}\tilde{q}^2v_4(ud^2x) + \tilde{q}^3v_4(d^3x)\right) \ , \end{aligned} \tag{3}$$

with x substituted by $\mathbb{S}_1$. Finally, we have

$$\begin{aligned} v_0(x) &= \frac{1}{1+r}\left(\tilde{p}v_1(ux) + \tilde{q}v_1(dx)\right) \ , \\ &= \frac{1}{(1+r)^4}\left(\tilde{p}^4v_4(u^4x) + 4\tilde{p}^3\tilde{q}v_4(u^3dx) + 6\tilde{p}^2\tilde{q}^2v_4(ud^2x) + 4\tilde{p}\tilde{q}^3v_4(ud^3x) + \tilde{q}^4v_4(d^4x)\right) \ , \end{aligned} \tag{4}$$

with x substituted by $\mathbb{S}_0$.

The general formula in the N-period binomial model is

$$v_n(x) = \frac{1}{(1+r)^{N-n}}\sum_{j=0}^{N-n}\binom{N-n}{j}\tilde{p}^{N-n-j}\tilde{q}^j v_N(u^{N-n-j}d^jx) \ . \tag{5}$$

with x substituted by $\mathbb{S}_n$.

0.0.2 Exercise 1b

We simulate a coin toss with probability p for heads and q for tails using the tower method of sampling discrete distributions. We essentially stack the probabilities p and q one on top of the other to form a tower of total height p + q = 1. We then generate a random number uniformly between 0 and 1. If the number generated is between 0 and p (the so-called ground floor of the tower), then we declare the outcome as heads. If the random number is between p and 1 (the first floor of the tower), then the outcome is declared as tails.

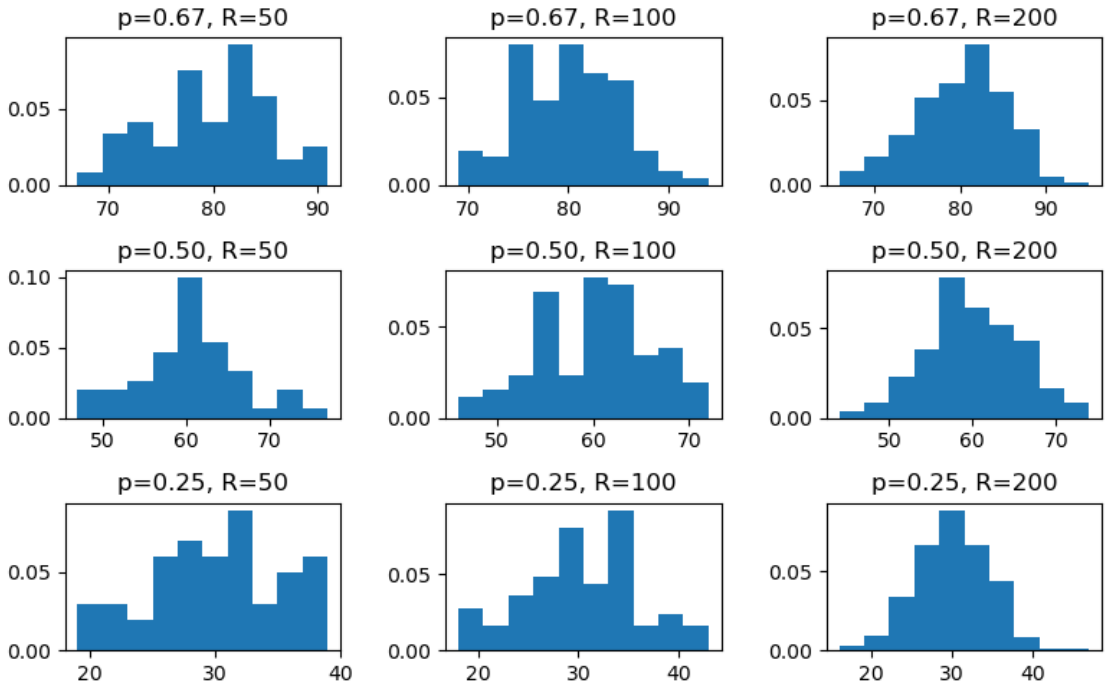
We toss the coin N = 120 times, and repeat this R = 50, 100, 200 times. We also choose three different values of the probabilities (p, q): (2/3, 1/3), (1/2, 1/2), (1/4, 3/4). We plot a histogram of the number of heads out of 120 tosses in each of the R trials. As the value of R increases, the histogram of heads becomes more and more peaked at the value p N.

[2]:
```
# initialize the Generator object. You can also random.random
rng = np.random.default_rng();
```

[3]:
```
def answer1b(p, q, N, R):
    randNum = rng.random(size=(R,N)) # generate an R x N array of random␣
 ↪numbers between 0 and 1
    randOutcome = (randNum < p).astype(int) # create a new R x N array of coin␣
 ↪toss outcomes
                                            # by setting an entry to 1 if the␣
 ↪corresponding entry
                                            # in randNum is less than p and 0␣
 ↪otherwise.
                                            # 1 is heads and 0 is tails
    randHeads = np.sum(randOutcome, axis = 1) # count the number of heads in␣
 ↪each of the repeated trials
                                              # by using np.sum with axis=1,␣
 ↪that is create the new array
                                              # randHeads along the column axis␣
 ↪(axis=1)

    return randHeads
```

[4]:
```
_, ax = plt.subplots(3,3, figsize=(8,5)) # create a subplot with 9 plots, one␣
 ↪for
                                         # each of the three probability values
                                         # and one for each of the three R␣
 ↪values

p = [2/3, 1/2, 1/4] # array of different probabilities
R = [50, 100, 200] # array of different repetitions

for i in np.arange(0,3): # loop over the different probability values
    for j in np.arange(0,3): # loop over the different number of repetitions
```

```
        ax[i,j].hist(answer1b(p[i], 1 - p[i], 120, R[j]), density=True) # plot the histogram in the i-th row and j-th column of the subplot
        ax[i,j].set_title(f'p={p[i]:.2f}, R={R[j]}')

plt.tight_layout() # to get appropriate spacing between the figures
```



0.0.3 Exercise 1c

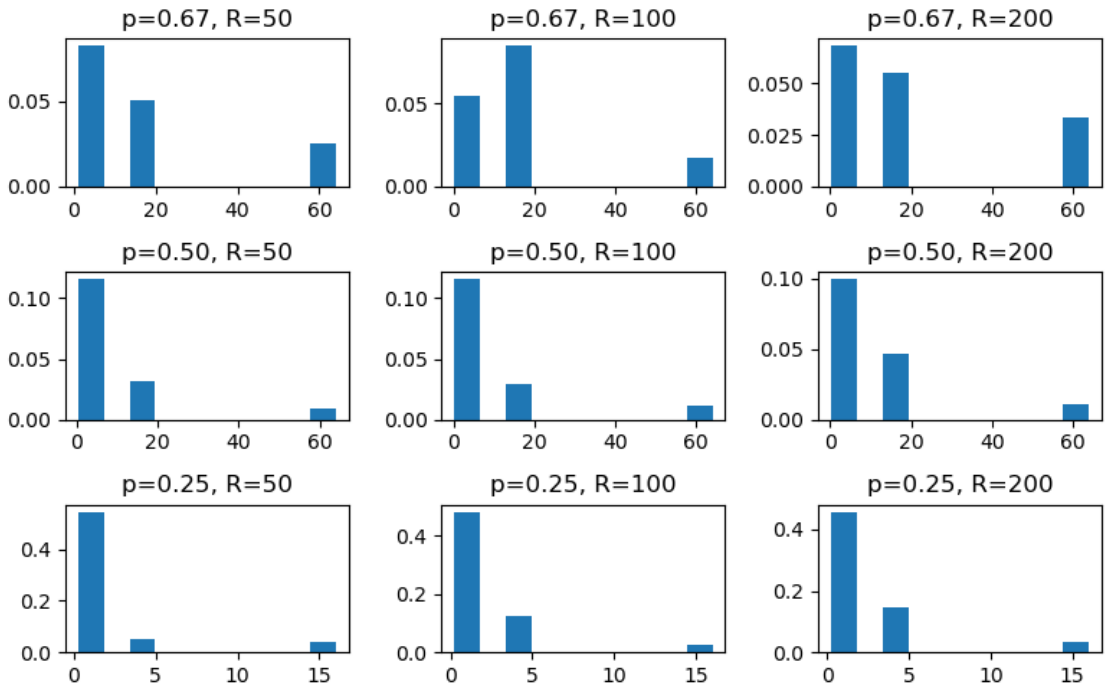
In this exercise, we simulate the binomial tree stock price process. We write a function which takes in the coin toss probabilities p, q, number of coin tosses N (number of periods of the binomial model), the number of repeated trials R for the Monte Carlo simulation, the up and down factors u and d, and the initial stock price. The function returns an array of length R, consisting of the R final stock price values at the end of each of the R trials of N coin tosses.

The number of coin tosses is N = 4, the up and down factors are 2 and 1/2 respectively and the initial stock price is 4.

[5]:
```
def answer1c(p, q, N, R, u, d, S0):
    randNum = rng.random(size=(R,N))
    randOutcome = (randNum < p).astype(int) # array of 1s and 0s, 1 for heads and 0 for tails
    randUpDown = np.where(randOutcome, u, d) # replace heads by u and tails by d
    stockPriceN = S0 * np.prod(randUpDown, axis=1) # take the product of all the u and d factors in each row of the array
```

```
                                                    # corresponding to the N␣
↪coin tosses. We get a column of R final stock price values

    return stockPriceN
```

[6]:
```
_, ax = plt.subplots(3,3, figsize=(8,5)) # create a subplot with 9 plots, one␣
↪for
                                         # each of the three probability values
                                         # and one for each of the three R␣
↪values

p = [2/3, 1/2, 1/4] # array of different probabilities
R = [50, 100, 200] # array of different repetitions

for i in np.arange(0,3): # loop over the different probability values
    for j in np.arange(0,3): # loop over the different number of repetitions
        ax[i,j].hist(answer1c(p[i], 1 - p[i], 4, R[j], 2, 1/2, 4),␣
↪density=True) # plot the histogram in the i-th row and j-th column of the␣
↪subplot
        ax[i,j].set_title(f'p={p[i]:.2f}, R={R[j]}')

plt.tight_layout() # to get appropriate spacing between the figures
```

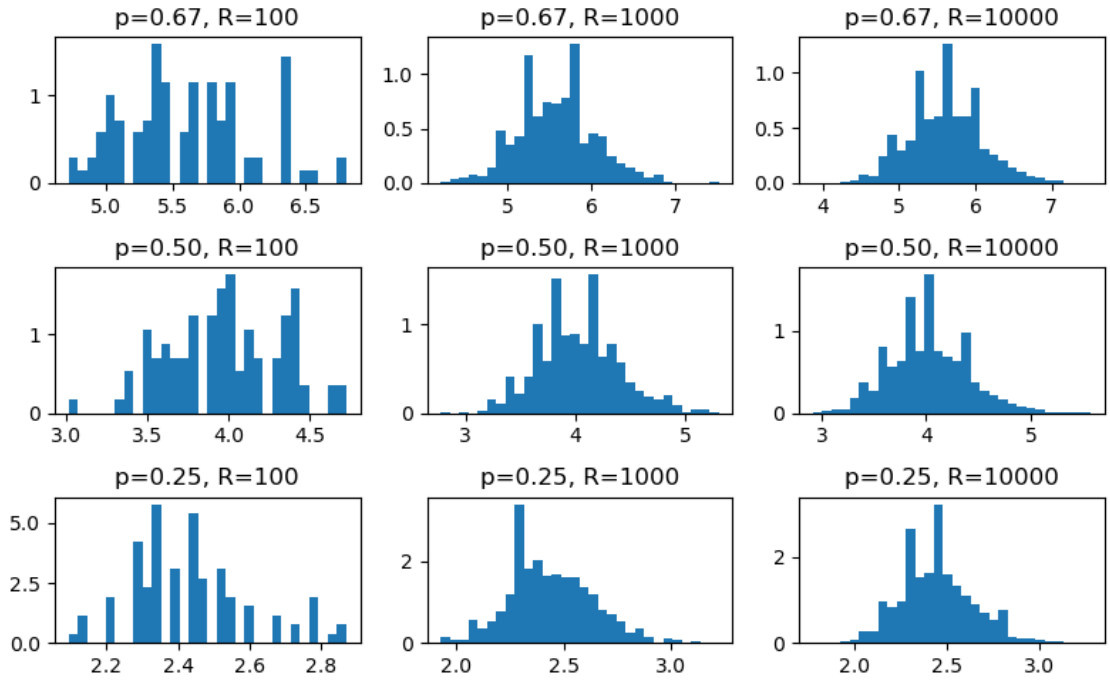


The number of possible stock prices at the end of four coin tosses is five, a very small number. Plotting the histograms as above is not very informative if there aren't many observations. A

histogram is truly useful when there are many observations. To achieve this, we have to have a binomial tree model of a much higher period, say N = 120. However, in this case, if we take u and d to be 2 and 1/2, then the final stock prices will be unimaginably large like 2^120 or small 2^{-120}. So, to see some reasonable final stock prices, we have to take u and d to be very close to 1. Let us choose u = 1 + 1/N and d = 1/u. In this case, u^N = (1 + 1/N)^N which is approximately 2.

Let us also increase the number of trials and consider R = 100, 1000, 10000, and the number of bins in the histogram to approx. 30. Then, we see a much smoother distribution of final stock price values due to the large number of coin tosses, large number of trials, and a higher number of bins.

[7]:
```
_, ax = plt.subplots(3,3, figsize=(8,5)) # create a subplot with 9 plots, one␣
↪for
                                         # each of the three probability values
                                         # and one for each of the three R␣
↪values

p = [2/3, 1/2, 1/4] # array of different probabilities
R = [100, 1000, 10000] # array of different repetitions
u = 1 + 1/120
d = 1 / u

for i in np.arange(0,3): # loop over the different probability values
    for j in np.arange(0,3): # loop over the different number of repetitions
        # plot the histogram in the i-th row and j-th column of the subplot
        ax[i,j].hist(answer1c(p[i], 1 - p[i], 120, R[j], u, d, 4),␣
↪density=True, bins=30)
        ax[i,j].set_title(f'p={p[i]:.2f}, R={R[j]}')

plt.tight_layout() # to get appropriate spacing between the figures
```



0.0.4 Exercise 1d

In this exercise, we estimate the fair price of a European call option using the risk-neutral pricing formula in the binomial model of asset pricing.

We write a function which takes in the the risk-free interest rate r, the number of periods of the binomial model N, the number of repeated trials R, the up factor u, the down factor $d = u^{-1}$, the initial stock price S_0 and the strike price K. The function returns the fair price of the option obtained as the mean of the discounted payoffs over the R trials.

The risk-free interest rate is $r = 1/4$, the up factor is $u = 2$ and the down factor is $d = 1/2|$. This gives the risk neutral probabilities $\tilde{p} = \tilde{q} = 1/2$. The number of periods is $N = 4$.

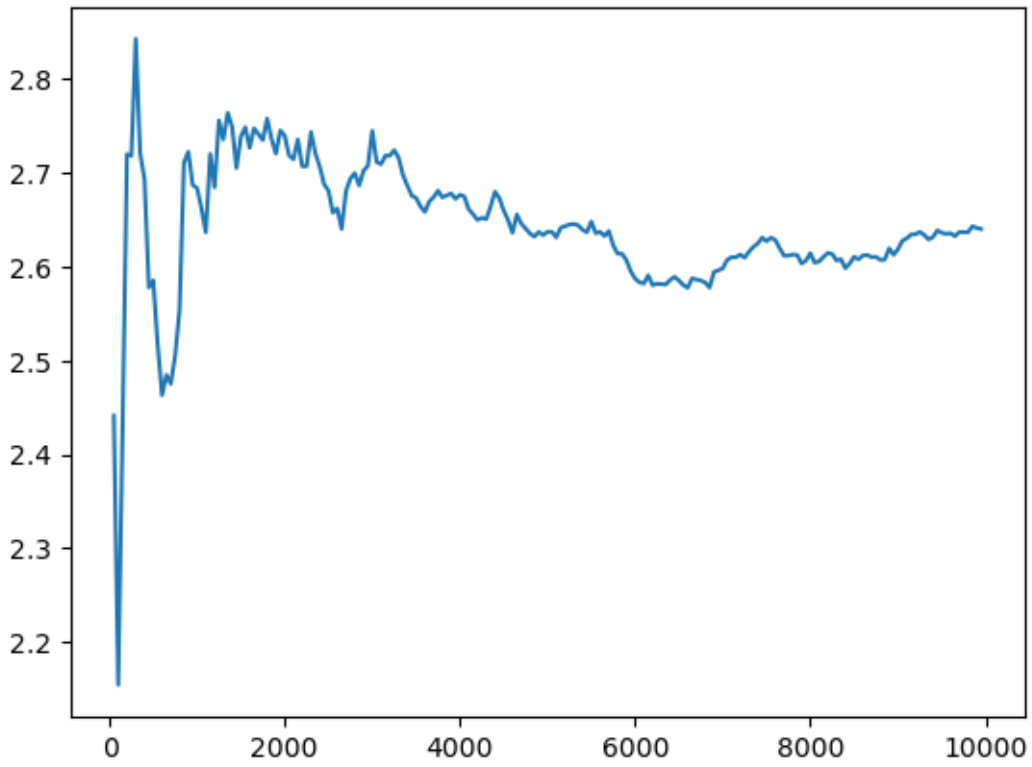
[8]:
```
def answer1d(r, N, R, u, d, S0, K):
    rng1 = np.random.default_rng(23)
    randNum = rng1.random(size=(R,N))
    p = (1 + r - d) / (u - d)
    randOutcome = (randNum <= p).astype(int)
    randUpDown = np.where(randOutcome, u, d)
    stockPriceN = S0 * np.prod(randUpDown, axis=1) # final stock price values
    payoffN = np.maximum(stockPriceN - K, 0) # final payoffs
    discPayoffN = payoffN / (1 + r)**N # discounted payoffs
    optionVal = np.mean(discPayoffN) # average of the discounted payoffs over␣
 ↪the R trials

    return optionVal
```

The exercise asks you to evaluate the value of the European call for the five values $R = 50, 100, 500, 1000, 10000$. Here, we vary R in steps of 50 from 50 to 10000 to see how the option price settles noisily to somewhere close to the theoretical value (this is the law of large numbers. Higher the number of repeated trials, closer the trial average is to the theoretical average using the theoretical probabilities). Usually, R must be very very large to come very close to the theoretical value.

[9]:
```
Rvals = np.arange(50,10000,50) # generate a list of values for the number of␣
↪repeated trials R from 50 to 10000 in steps of 50
```

[10]:
```
# compute the value of the option for each of the values of R generated above
optionVals = np.array([answer1d(1/4, 4, x, 2, 1/2, 4, 5) for x in Rvals])
# plot the option value versus the corresponding value of R
plt.plot(Rvals,optionVals)
```

[10]:
```
[<matplotlib.lines.Line2D at 0x7f68e9f91950>]
```



[11]:
```
# the value at the end of 10000 trials
optionVals[-1]
```

[11]:
```
np.float64(2.640046954773869)
```

The theoretical value of the option is computed using the usual risk-neutral pricing formula involv-

ing expectation value of the discounted payoff w.r.t. the risk-neutral probability measure. This is given by the formula

$$v_0(x) = \frac{1}{(1+r)^N} \sum_{i=0}^{N} \binom{N}{i} \tilde{p}^{N-i} \tilde{q}^i v_4(u^{N-i} d^i x) \ . \tag{6}$$

We can compute this sum using a quick list comprehension.

[12]:
```
binomVals = np.array([(1 / (1+(1/4))**4) * scipy.special.binom(4,i) * ((1/
↪2)**(4)) * np.maximum(4*(2**(4-2*i))-5, 0) for i in [0,1,2,3]])
binomVals.sum() # the theoretical average using the risk-neutral probability␣
↪measure
```

[12]:
```
np.float64(2.6368)
```

Alternately, we can define a function to the above job.

[13]:
```
def optionPriceTh(r, u, d, N, S0, K):
    p = (1 + r - d) / (u - d)
    q = (u - 1 - r) / (u - d)
    val = 0
    for i in np.arange(0,N+1,1):
        val += (1 / (1+r)**N) * scipy.special.binom(N,i) * (p**(N-i)) * (q**i) *␣
↪np.maximum(S0*(u**(N-i))*(d**i)-K, 0)
    return val
```

[14]:
```
optionPriceTh(1/4, 2, 1/2, 4, 4, 5)
```

[14]:
```
np.float64(2.6368)
```

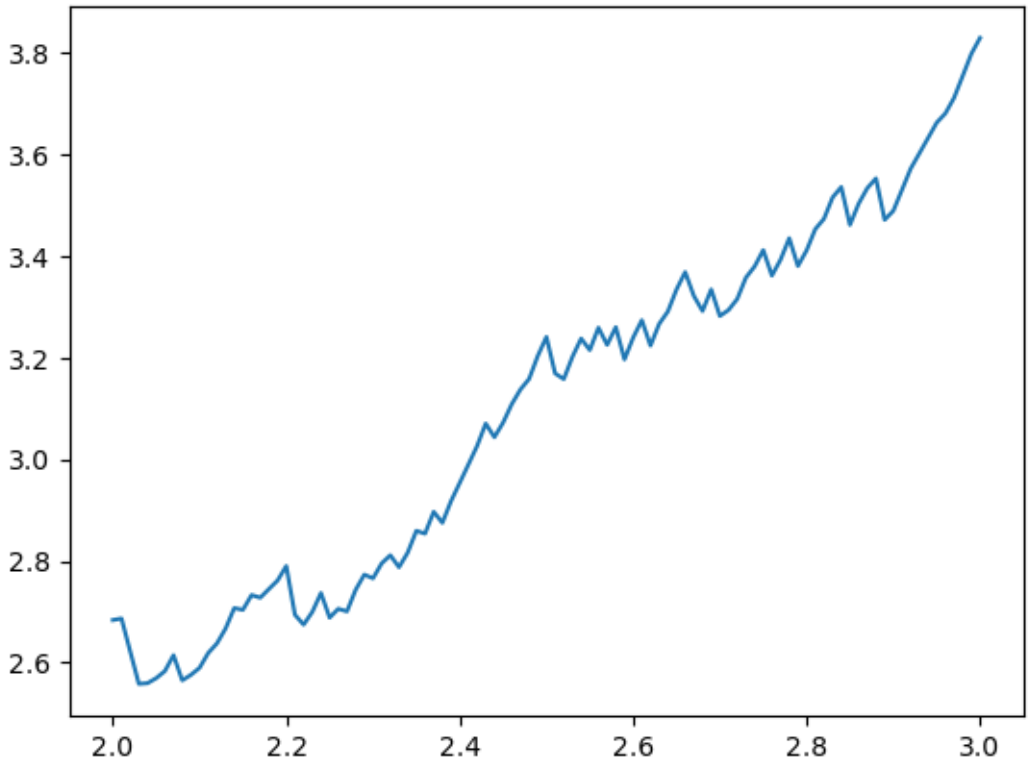
0.0.5 Exercise 1e

In this exercise, we use the same function as in Exercise 1d to compute the fair value of the option at time 0. However, we stick to 1000 trials and study the variation in the option value as the volatility is changed i.e., as the value of u is changed from 2 to 3. Care must be taken to use the correct risk-neutral probabilities for each of the values of u.

[15]:
```
uVals = np.arange(2,3.01,0.01)
```

[16]:
```
optionVals = np.array([answer1d(1/4, 4, 1000, x, 1/x, 4, 5) for x in uVals])
plt.plot(uVals,optionVals)
```

[16]:
```
[<matplotlib.lines.Line2D at 0x7f68e9e18410>]
```



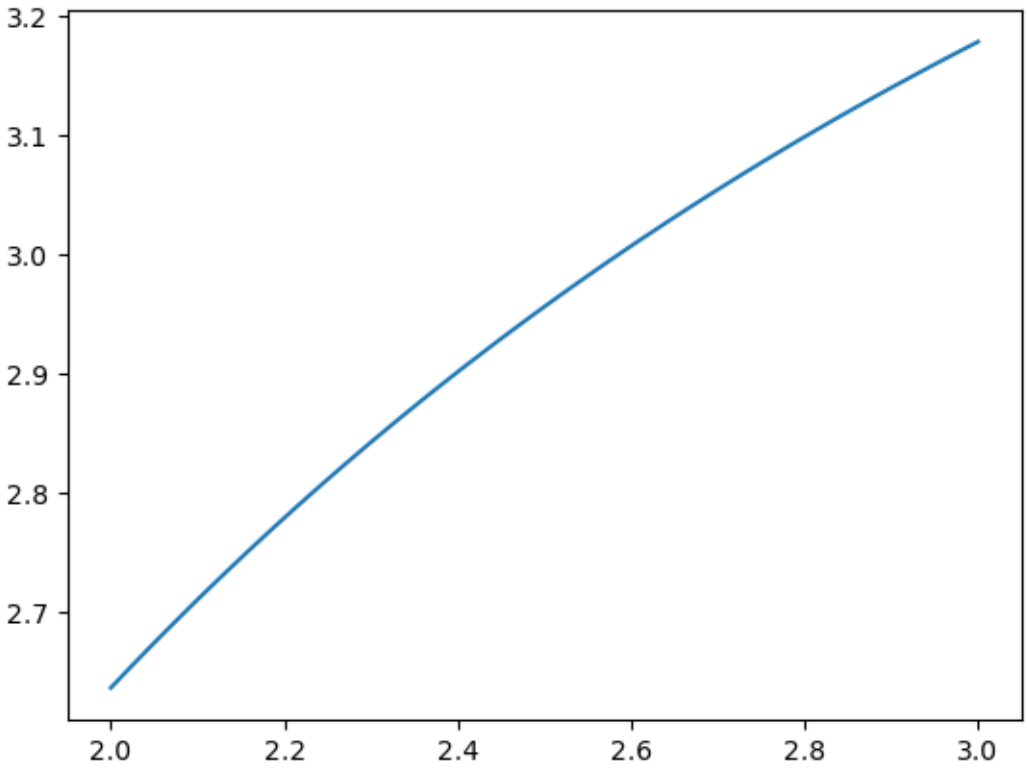
0.0.6 Exercise 1f

In this exercise, we compute the theoretical dependence of the option price on the up factor.

[17]:
```
optionValsTh = np.array([optionPriceTh(1/4, x, 1/x, 4, 4, 5) for x in uVals])
```

[18]:
```
plt.plot(uVals,optionValsTh)
```

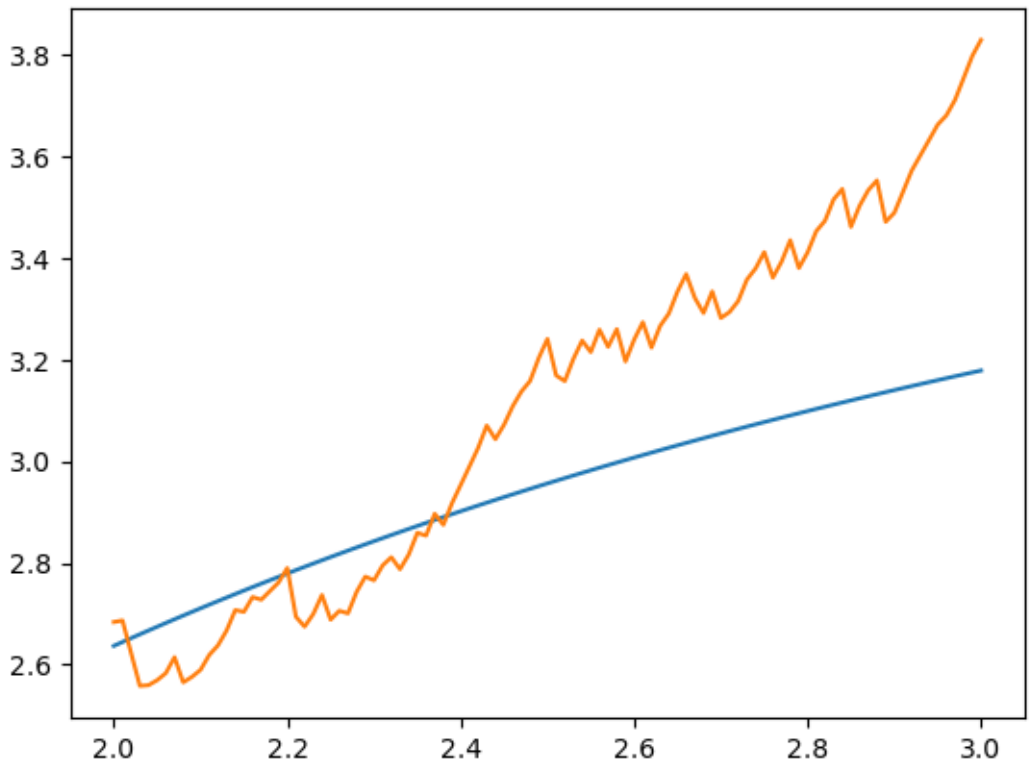
[18]:
```
[<matplotlib.lines.Line2D at 0x7f68e9e76990>]
```



This is the theoretical dependence of the option value on the up factor (which encodes the volatility of the stock). Let us next compare the numerical dependence computed in Exercise 1e with this theoretical dependence.

[19]:
```
plt.plot(uVals,optionValsTh)
plt.plot(uVals,optionVals)
```

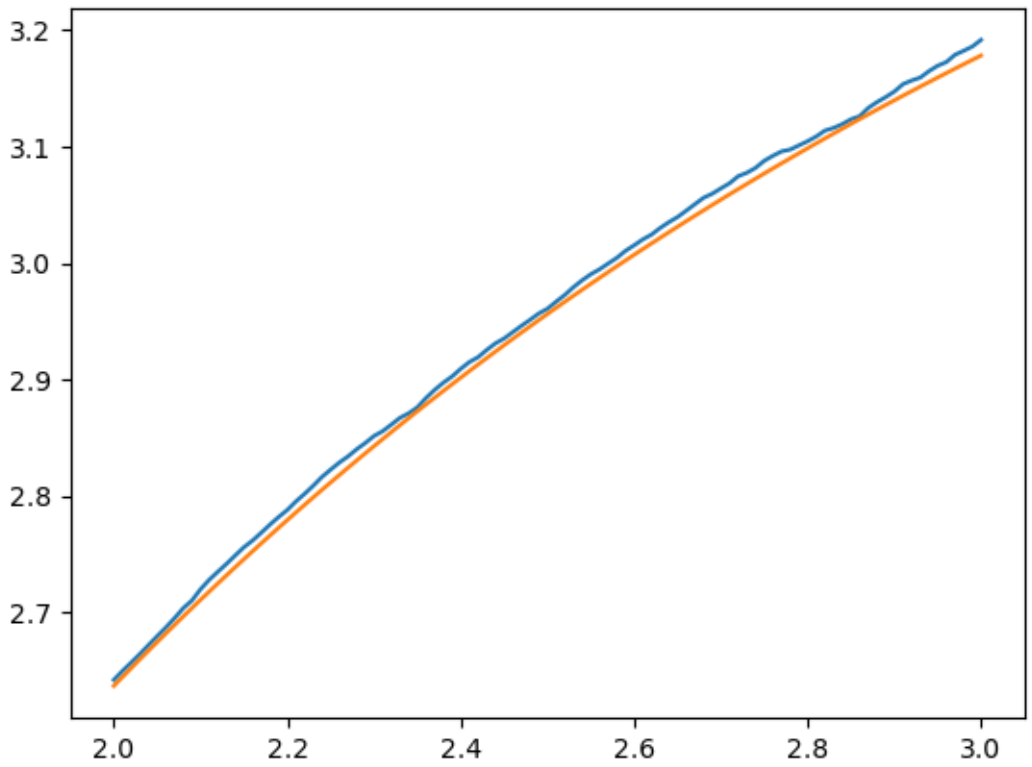
[19]:
```
[<matplotlib.lines.Line2D at 0x7f68e9d04910>]
```



The numerical dependence of the option value on the up factor that we computed in Exercise 1e is very jagged for $R = 1000$. It is not easy to compare this with the theoretical curve. Let us try for $R = 10^6$ (1 million):

[20]:
```
# Exercise 1d with R = 10^6 instead of R = 10^3
optionVals1 = np.array([answer1d(1/4, 4, 10**6, x, 1/x, 4, 5) for x in uVals])
plt.plot(uVals,optionVals1)
plt.plot(uVals,optionValsTh)
```

[20]:
```
[<matplotlib.lines.Line2D at 0x7f68e9d62fd0>]
```



Clearly, the numerical data for 1 million repetitions is much better as expected since higher the number of repetitions, closer the theoretical probabilities are to the numerical ones.

0.0.7 Exercise 1g

In this exercise, the time evolution of the option price is computed using the conditional expectation formula $v_n(\mathbb{S}_n) = \tilde{\mathbb{E}}\left[\frac{v_N(\mathbb{S}_N)}{(1+r)^{N-n}} \,\middle|\, \mathcal{A}_n\right]$. We take the up factor to be 2, down factor to be 1/2, risk-free interest rate 1/4.

[21]:
```
# First we compute the possible stock prices at each time instant n between 1␣
↪and N.
stock1 = np.array([4*(2**(1-i))*((1/2)**i) for i in np.arange(0,2,1)])
stock2 = np.array([4*(2**(2-i))*((1/2)**i) for i in np.arange(0,3,1)])
stock3 = np.array([4*(2**(3-i))*((1/2)**i) for i in np.arange(0,4,1)])
stock4 = np.array([4*(2**(4-i))*((1/2)**i) for i in np.arange(0,5,1)])
```

We now use the option pricing function of Exercise 1d answer1d(r, N, R, u, d, S0, K) with $r = 1/4$, N replaced by $N - n$, $R = 1000$, $u = 2$, $d = 1/2$, $S0$ replaced by one of the stock prices in the array stockn, $K = 5$

[22]:
```
optionPrice0 = answer1d(1/4, 4, 1000, 2, 1/2, 4, 5) # Checking again the option␣
↪price at t = 0
```

```
# Two option prices for two possible values of S1
optionPrice1 = np.array([answer1d(1/4, 4-1, 1000, 2, 1/2, stock1[x], 5) for x␣
 ↪in np.arange(0,2,1)])
# Three option prices for two possible values of S2
optionPrice2 = np.array([answer1d(1/4, 4-2, 1000, 2, 1/2, stock2[x], 5) for x␣
 ↪in np.arange(0,3,1)])
# Four option prices for two possible values of S3
optionPrice3 = np.array([answer1d(1/4, 4-3, 1000, 2, 1/2, stock3[x], 5) for x␣
 ↪in np.arange(0,4,1)])
# Five option prices for two possible values of S4 (these are the payoffs at␣
 ↪expiry)
optionPrice4 = np.array([answer1d(1/4, 4-4, 1000, 2, 1/2, stock4[x], 5) for x␣
 ↪in np.arange(0,5,1)])
```

[23]:
```
optionPrice0
```

[23]: np.float64(2.6836992)

Let us calculate the theoretical prices using the conditional expectation value formula with the theoretical risk-neutral probabilities.

[24]:
```
optionPriceTh0 = optionPriceTh(1/4, 2, 1/2, 4, 4, 5)
optionPriceTh1 = np.array([optionPriceTh(1/4, 2, 1/2, 4-1, x, 5) for x in␣
 ↪stock1])
optionPriceTh2 = np.array([optionPriceTh(1/4, 2, 1/2, 4-2, x, 5) for x in␣
 ↪stock2])
optionPriceTh3 = np.array([optionPriceTh(1/4, 2, 1/2, 4-3, x, 5) for x in␣
 ↪stock3])
optionPriceTh4 = np.array([optionPriceTh(1/4, 2, 1/2, 4-4, x, 5) for x in␣
 ↪stock4])
```

[25]:
```
optionPrice0 - optionPriceTh0
```

[25]: np.float64(0.04689919999999992)

[26]:
```
optionPrice1 - optionPriceTh1
```

[26]: array([-0.35328 , -0.073216])

[27]:
```
optionPrice3 - optionPriceTh3
```

[27]: array([-0.2688, -0.0616, 0.   , 0.   ])

[28]:
```
optionPrice4 - optionPriceTh4
```

[28]: array([0., 0., 0., 0., 0.])

As we can see, the difference between the Monte Carlo and theoretical answers are small. They can be made more accurate with repetitions, or by other methods.

0.0.8 Exercise 1h

In this exercise, we calculate the Δ-hedging fractions $\Delta_n(\mathbb{S}_n)$. We first make a dictionary out of the stock prices at time $n+1$ and the option prices at time $n+1$. This represents the function v_{n+1} as a function of $\mathbb{S}_{n+1}$.

[29]:
```
dict1 = dict(zip(stock1, optionPrice1))
dict2 = dict(zip(stock2, optionPrice2))
dict3 = dict(zip(stock3, optionPrice3))
dict4 = dict(zip(stock4, optionPrice4))
```

We then calculate the Δ_n using the formula $\Delta_n = \frac{v_{n+1}(u\mathbb{S}_n) - v_{n+1}(d\mathbb{S}_n)}{u\mathbb{S}_n - d\mathbb{S}_n}$.

[30]:
```
Delta0 = np.array([(dict1[2*x] - dict1[x/2])/(2*x - x/2) for x in [4]])
Delta1 = np.array([(dict2[2*x] - dict2[x/2])/(2*x - x/2) for x in stock1])
Delta2 = np.array([(dict3[2*x] - dict3[x/2])/(2*x - x/2) for x in stock2])
Delta3 = np.array([(dict4[2*x] - dict4[x/2])/(2*x - x/2) for x in stock3])
```

[31]:
```
Delta0
```

[31]:
```
array([0.81732267])
```

[32]:
```
Delta1
```

[32]:
```
array([0.92410667, 0.58197333])
```

[33]:
```
Delta2
```

[33]:
```
array([0.9747    , 0.72306667, 0.        ])
```

[34]:
```
Delta3
```

[34]:
```
array([1.        , 0.91666667, 0.        , 0.        ])
```

One thing to notice is that Δ_n is always less than 1. This can be shown by looking at the option prices as a function of stock prices carefully.

0.0.9 Exercise 1i

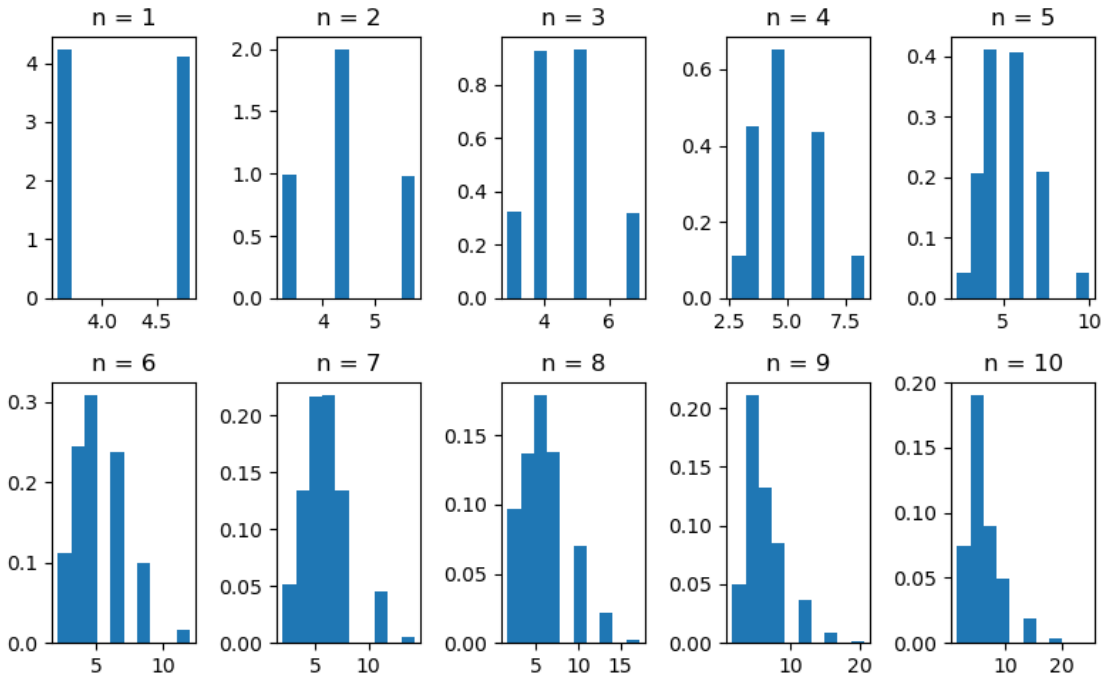
In this exercise, we make a minor modification of the code for Exercise 1c to keep track of all stock price values, not just the final one at time N. If we use the risk neutral measure, it does not matter if ud > 1, ud < 1 or ud $= 1$. The stock price process has an average drift to the right since it gets a mean rate of return equal to the risk free interest rate r per period. The discounted stock price histograms do not drift to the right since the average rate of return of the discounted stock is zero under the risk neutral probabilities.

Just to illustrate the point, we look at the case ud > 1 with u $= 1.2$, d $= 0.9$, r $= 1/20$.

[166]:
```
def answer1i(r, N, R, u, d, S0):
    rng = np.random.default_rng(seed=23)
```

```
    randNum = rng.random(size=(R,N))
    p = (1 + r - d)/(u - d)
    randOutcome = (randNum < p).astype(int) # array of 1s and 0s, 1 for heads␣
 ↪and 0 for tails
    randUpDown = np.where(randOutcome, u, d) # replace heads by u and tails by d
    stockPriceN = S0 * np.cumprod(randUpDown, axis=1) # take the running or␣
 ↪cumulative product of all the u and d factors in each row
                                                      # of the array␣
 ↪corresponding to the N coin tosses.
                                                      # We get an R x N array␣
 ↪of stock price values

    return stockPriceN
```

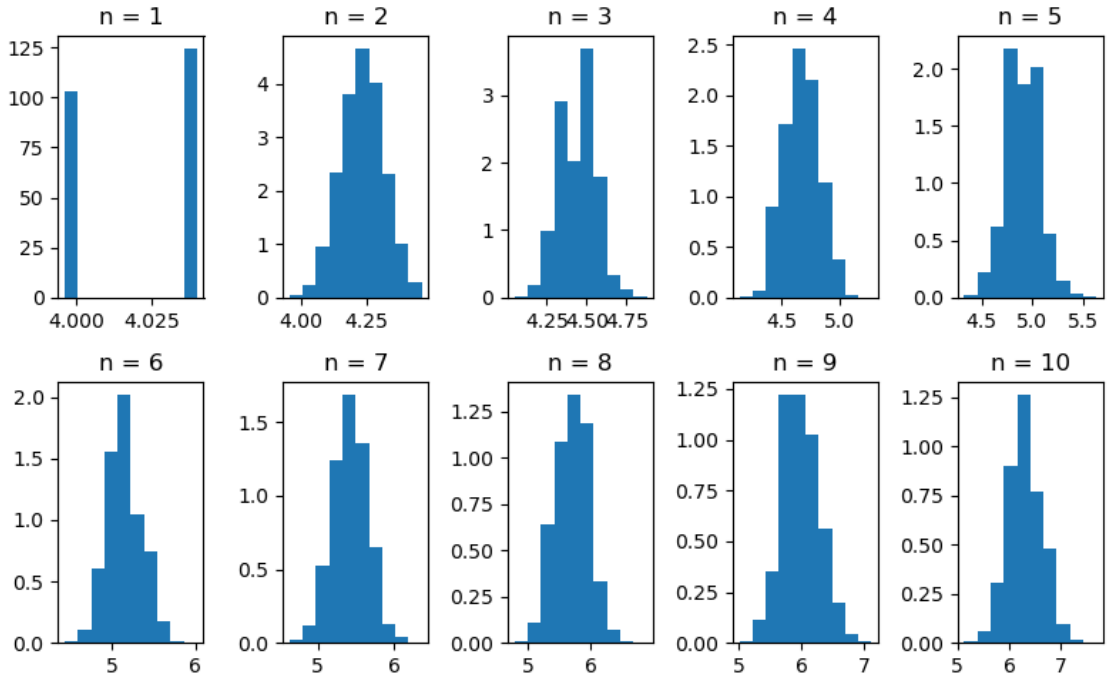
[167]:
```
_, ax = plt.subplots(2,5, figsize=(8,5)) # create a subplot with 10 plots, one␣
 ↪for
                                         # each of the 10 time instants.

stockPriceEvol = answer1i(1/20, 10, 10000, 1.2, 0.9, 4)

for i in np.arange(0,5,1): # loop over the first five time instants i.e. first␣
 ↪5 columns in the R x N array
    ax[0,i].hist(stockPriceEvol[:,i], density=True) # plot the histogram in the␣
 ↪first row and i-th column of the subplot
    #ax[0,i].set_xscale('log')
    ax[0,i].set_title(f'n = {i+1}')

for i in np.arange(5,10,1): # loop over the second five time instants i.e.␣
 ↪second 5 columns in the R x N array
    ax[1,i-5].hist(stockPriceEvol[:,i], density=True) # plot the histogram in␣
 ↪the second row and i-th column of the subplot
    #ax[1,i-5].set_xscale('log')
    ax[1,i-5].set_title(f'n = {i+1}')

plt.tight_layout() # to get appropriate spacing between the figures
```



It may be clear from the above histograms that they drift to the right slightly over the 10 periods. To show this drift more clearly, we can choose N = 100, and u = 1.01, d = 0.999 so that ud > 1, and r = 1/200. Instead of plotting the histograms for all 100 periods, we show 10 histograms in steps of 10 from 0 to 100.

[168]:
```
_, ax = plt.subplots(2,5, figsize=(8,5)) # create a subplot with 10 plots, one␣
↪for
                                         # each of the 10 time instants.

stockPriceEvol = answer1i(1/200, 100, 10000, 1.01, 0.999, 4)

for i in np.arange(0,5,1): # loop over the first five time instants i.e. first␣
↪5 columns in the R x N array
    ax[0,i].hist(stockPriceEvol[:,10*i], density=True) # plot the histogram in␣
↪the first row and i-th column of the subplot
    #ax[0,i].set_xscale('log')
    ax[0,i].set_title(f'n = {i+1}')

for i in np.arange(5,10,1): # loop over the second five time instants i.e.␣
↪second 5 columns in the R x N array
    ax[1,i-5].hist(stockPriceEvol[:,10*i], density=True) # plot the histogram␣
↪in the second row and i-th column of the subplot
    #ax[1,i-5].set_xscale('log')
    ax[1,i-5].set_title(f'n = {i+1}')
```

```
plt.tight_layout() # to get appropriate spacing between the figures
```



It is a bit more clear now that the histograms drift towards the right due to the rate of return being the risk-free interest rate.

0.0.10 Exercise 1j

We modify the code of Exercise 1i to include different probabilities, including the risk-neutral probabilities. It is best if we plot the histograms of the discounted stock prices since these histograms do not drift over time under the risk-neutral probabilities. It will be easy to compare the results of the cases where the probabilities are not risk-neutral.

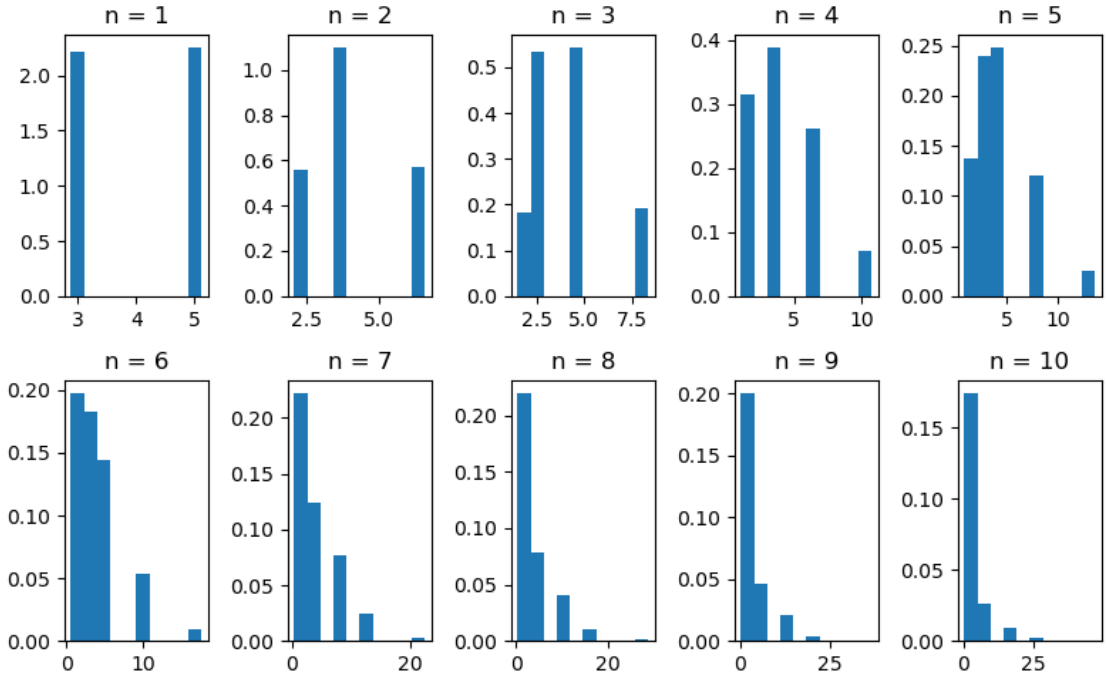
[182]:
```
(1 + 1/24 - 3/4)/(4/3 - 3/4)
```

[182]:
```
0.5000000000000002
```

[175]:
```
def answer1j(r, p, N, R, u, d, S0):
    randNum = rng.random(size=(R,N))
    randOutcome = (randNum <= p).astype(int) # array of 1s and 0s, 1 for heads␣
↪and 0 for tails
    randUpDown = np.where(randOutcome, u/(1+r), d/(1+r)) # replace heads by u/
↪(1+r) and tails by d/(1+r) to account for discounting
    discStockPriceN = S0 * np.cumprod(randUpDown, axis=1) # take the running or␣
↪cumulative product of all the u and d factors in each row
```

```
                                                    # of the array␣
 ↪corresponding to the N coin tosses.
                                                    # We get an R x N array␣
 ↪of stock price values

    return discStockPriceN
```

[181]:
```
_, ax = plt.subplots(2,5, figsize=(8,5)) # create a subplot with 10 plots, one␣
 ↪for
                                         # each of the 10 time instants.

stockPriceEvol1 = answer1j(1/24, 1/2, 10, 10000, 4/3, 3/4, 4)

for i in np.arange(0,5,1): # loop over the first five time instants i.e. first␣
 ↪5 columns in the R x N array
    ax[0,i].hist(stockPriceEvol1[:,i], density=True) # plot the histogram in␣
 ↪the first row and i-th column of the subplot
    ax[0,i].set_title(f'n = {i+1}')

for i in np.arange(5,10,1): # loop over the second five time instants i.e.␣
 ↪second 5 columns in the R x N array
    ax[1,i-5].hist(stockPriceEvol1[:,i], density=True) # plot the histogram in␣
 ↪the second row and i-th column of the subplot
    ax[1,i-5].set_title(f'n = {i+1}')

plt.tight_layout() # to get appropriate spacing between the figures
```

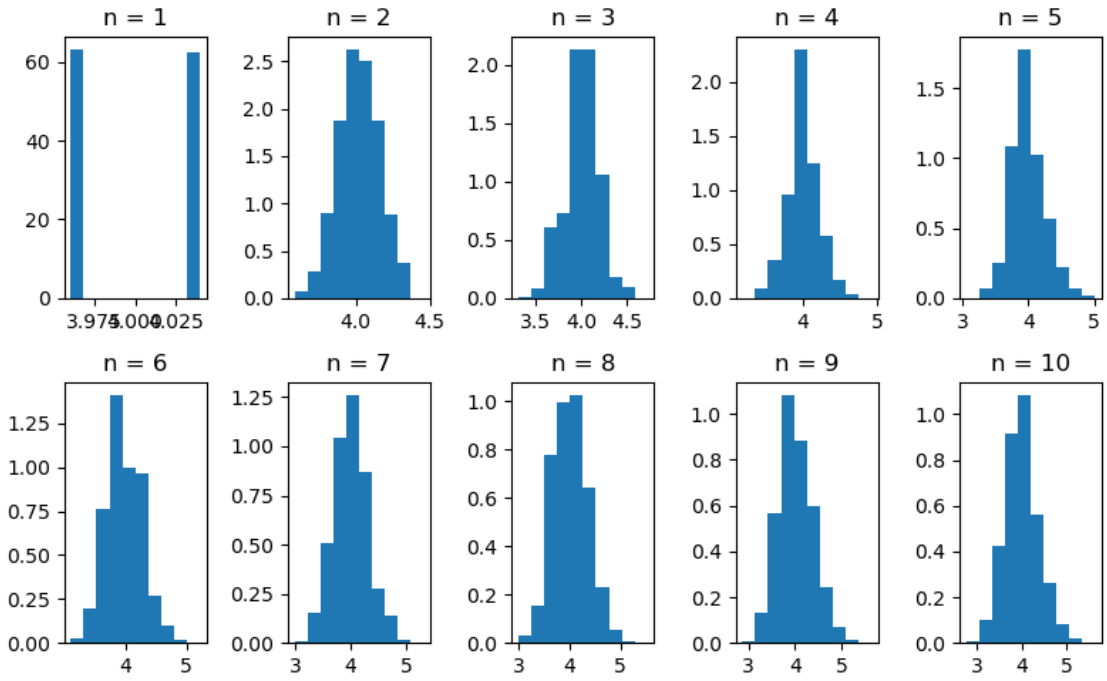


The peak of the histogram is around the same value, but again it is difficult to see due to the small number of periods, which in turn gives only a scattering of stock price values. To see a fat histogram, we need lots of different stock price values. One way to do this is to increase the number of periods, while keeping the u and d factors close enough to 1 so that the stock prices do not exponentially blow up or decrease. Let us try with u = 1.01, d = 1/u, r = 1/20200 which ensures that the risk neutral probabilities are p = q = 1/2.

[183]:
```
(1 + 1/20200 - 100/101)/(101/100 - 100/101)
```

[183]:
```
0.49999999999999445
```

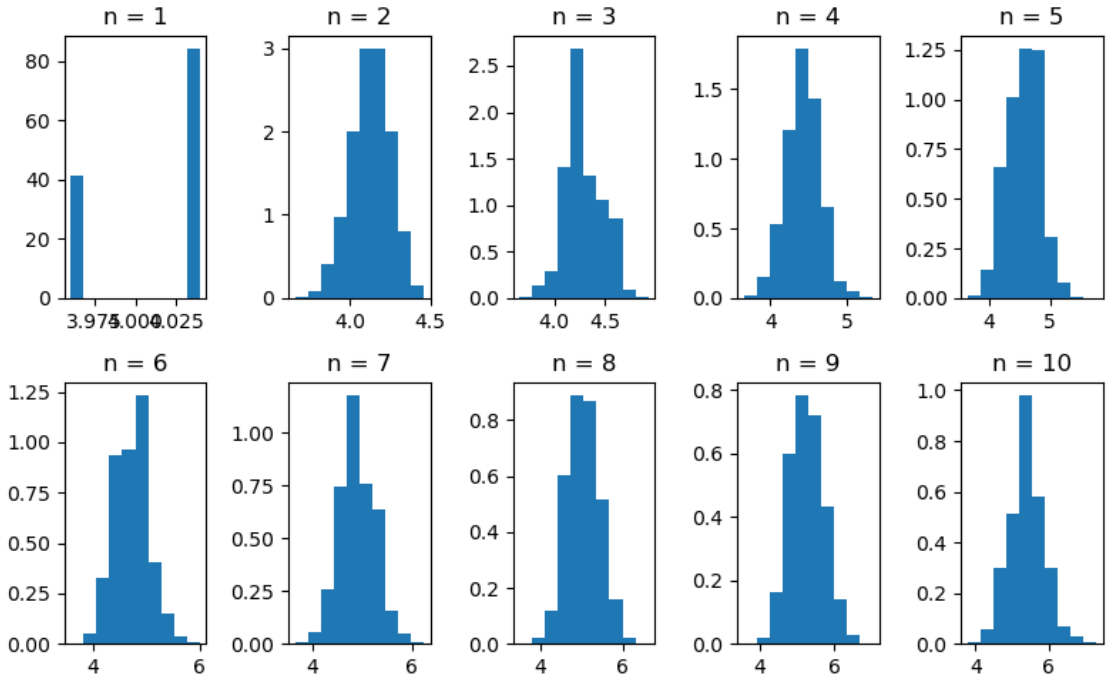
[185]:
```
_, ax = plt.subplots(2,5, figsize=(8,5)) # create a subplot with 10 plots, one␣
 ↪for
                                         # each of the 10 time instants.

stockPriceEvol1 = answer1j(1/20200, 1/2, 100, 10000, 1.01, 1/1.01, 4)

for i in np.arange(0,5,1): # loop over the first five time instants i.e. first␣
 ↪5 columns in the R x N array
    ax[0,i].hist(stockPriceEvol1[:,10*i], density=True) # plot the histogram in␣
 ↪the first row and i-th column of the subplot
    ax[0,i].set_title(f'n = {i+1}')

for i in np.arange(5,10,1): # loop over the second five time instants i.e.␣
 ↪second 5 columns in the R x N array
    ax[1,i-5].hist(stockPriceEvol1[:,10*i], density=True) # plot the histogram␣
 ↪in the second row and i-th column of the subplot
    ax[1,i-5].set_title(f'n = {i+1}')

plt.tight_layout() # to get appropriate spacing between the figures
```



It is clear that the histogram does not drift and is centred around the expected value S0 = 4.

Next, we choose different probabilities but keep the same up and down factors, risk-free interest rate and the number of periods: u = 1.01, d = 1/1.01, r = 1/20200, N = 100

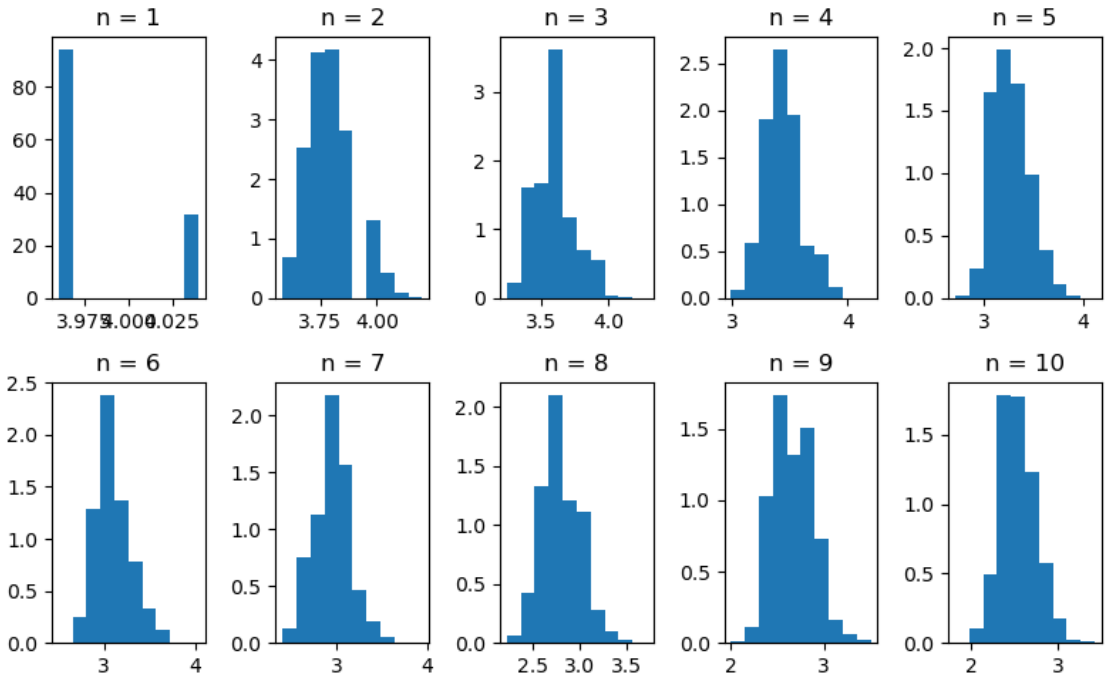
[186]:
```
_, ax = plt.subplots(2,5, figsize=(8,5)) # create a subplot with 10 plots, one␣
↪for
                                         # each of the 10 time instants.

stockPriceEvol2 = answer1j(1/20200, 2/3, 100, 10000, 1.01, 1/1.01, 4)

for i in np.arange(0,5,1): # loop over the first five time instants i.e. first␣
↪5 columns in the R x N array
    ax[0,i].hist(stockPriceEvol2[:,10*i], density=True) # plot the histogram in␣
↪the first row and i-th column of the subplot
    ax[0,i].set_title(f'n = {i+1}')

for i in np.arange(5,10,1): # loop over the second five time instants i.e.␣
↪second 5 columns in the R x N array
    ax[1,i-5].hist(stockPriceEvol2[:,10*i], density=True) # plot the histogram␣
↪in the second row and i-th column of the subplot
    ax[1,i-5].set_title(f'n = {i+1}')

plt.tight_layout() # to get appropriate spacing between the figures
```



As we can see, since p = 2/3 is greater than q = 1/3, the tendency is greater for the stock price to rise as time progresses. There is an upward drift in the discounted stock price. Hence, this is a submartingale.

Next, we choose p = 1/4 and q = 3/4, with the same choices for the other parameters as above.

[188]:
```
_, ax = plt.subplots(2,5, figsize=(8,5)) # create a subplot with 10 plots, one␣
↪for
                                         # each of the 10 time instants.

stockPriceEvol3 = answer1j(1/20200, 1/4, 100, 10000, 1.01, 1/1.01, 4)

for i in np.arange(0,5,1): # loop over the first five time instants i.e. first␣
↪5 columns in the R x N array
    ax[0,i].hist(stockPriceEvol3[:,10*i], density=True) # plot the histogram in␣
↪the first row and i-th column of the subplot
    ax[0,i].set_title(f'n = {i+1}')

for i in np.arange(5,10,1): # loop over the second five time instants i.e.␣
↪second 5 columns in the R x N array
    ax[1,i-5].hist(stockPriceEvol3[:,10*i], density=True) # plot the histogram␣
↪in the second row and i-th column of the subplot
    ax[1,i-5].set_title(f'n = {i+1}')

plt.tight_layout() # to get appropriate spacing between the figures
```



The histograms drift to the left, so the average rate of return has a tendency to decrease. Thus, this is a supermartingale.